

# Accelerating configuration of Reconfigurable Intelligent Surfaces through a hardware-enhanced deep learning approach

Rubén Padial-Allué <sup>a,\*,</sup>, Alberto Martín-Martín <sup>a,b,</sup>, Antonio García <sup>a,c,</sup>,  
Encarnación Castillo <sup>a,c,</sup>, José David Fernández-Rodríguez <sup>d,e,</sup>,  
Marcos Baena-Molina <sup>f,</sup>, Luis Parrilla <sup>a,c,</sup>

<sup>a</sup> Department Electronics and Computer Technology, University of Granada, Campus Universitario Fuentenueva, Granada, 18071, Spain

<sup>b</sup> Safran Electronics & Defense, Granada, 18014, Spain

<sup>c</sup> CITIC-UGR, University of Granada, Granada, 18071, Spain

<sup>d</sup> ITIS Software, University of Málaga, c/Arquitecto Francisco Peñalosa 18, Málaga, 29010, Spain

<sup>e</sup> Biomedic Research Institute of Málaga (IBIMA), c/Doctor Miguel Díaz Recio, 28, Málaga, 29010, Spain

<sup>f</sup> Department Signal Theory, Telematics and Communications, CITIC-UGR, University of Granada, Granada, 18071, Spain

## ARTICLE INFO

### Keywords:

6G  
Reconfigurable Intelligent Surfaces  
Intelligent reflecting surfaces  
Artificial intelligence  
Neural networks  
Field-Programmable Gate Arrays (FPGA)

## ABSTRACT

The dawn of Reconfigurable Intelligent Surfaces (RIS) promises to revolutionize wireless communication by enabling dynamic control of the propagation of electromagnetic waves. However, the practical implementation of RIS demands sophisticated configuration strategies to unlock their full potential. This paper presents a hardware implementation of a Deep Learning (DL) approach to the configuration of RIS, addressing both the complexity and efficiency of the configuration process. A deep learning-based algorithm, designed to optimize the phase shifts of RIS elements and shown to enhance signal quality and system performance, is implemented on two dedicated hardware platforms based on Field-Programmable Gate Arrays (FPGA), which leads to real-time processing and adaptability. This approach leverages the inherent parallelism of FPGAs to accelerate the computationally intensive tasks associated with deep learning inference. As a matter of fact, it is possible to achieve more than 18.000 configurations per second, thus ensuring rapid and efficient RIS configuration with a novel approach within this field.

## 1. Introduction

The rapid advancement of wireless communication technology necessitates innovative solutions to meet the ever-increasing demands for higher data rates, improved coverage, and energy efficiency. On June 2018, after years of research and development, the initial commercial roll-out of the 5th generation (5G) mobile communication standard was successfully completed. The new generation is already delivering speeds 10 times faster than its predecessor, with up to 20-Gbps peak data rates [1]. According to the Global System for Mobile Communications Association (GSMA), a non-profit organization representing the interest of mobile networks worldwide, 5G is on track to account for 15% of the global mobile industry by 2025 [2].

However, it has become clear during the standardization process of 5G wireless networks that no single enabling technology can meet all the application requirements of 5G [3]. Consequently, researchers have already begun exploring technologies beyond 5G

\* Corresponding author.

E-mail address: [rubenpadial@ugr.es](mailto:rubenpadial@ugr.es) (R. Padial-Allué).

<https://doi.org/10.1016/j.compeleceng.2026.111101>

Received 26 November 2025; Received in revised form 2 February 2026; Accepted 4 March 2026

0045-7906/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

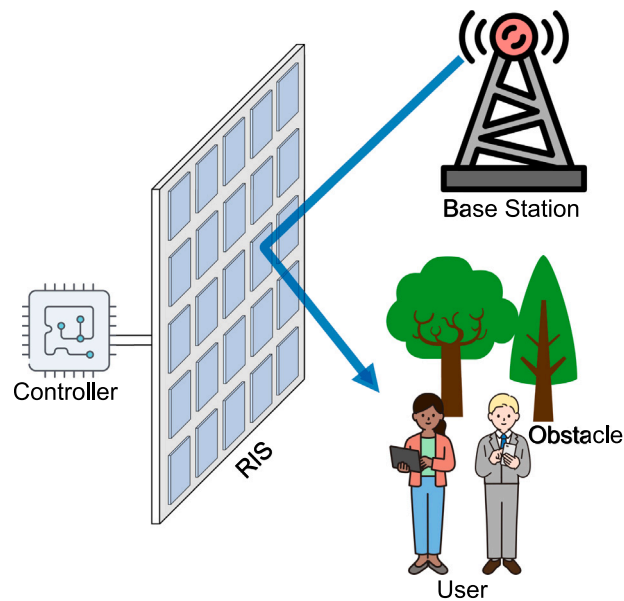


Fig. 1. Use of RIS to create a virtual LOS.

to improve data rates, reduce latency, and boost energy efficiency. To meet these stringent requirements, transmission technologies must advance further, relying on the adoption of new and innovative technologies.

RIS (Reconfigurable Intelligent Surfaces), alternatively referred to in the literature as Intelligent Reflecting Surfaces (IRS) and Large Intelligent Surfaces (LIS), consist of a large number of low-cost electromagnetic (EM) passive reflecting elements, each capable of adjusting its phase and amplitude in response to a control signal [4–6]. This unique capability enables RIS to actively shape and steer the wireless environment, thereby facilitating improved signal quality, reduced interference and enhanced spectral efficiency. Unlike traditional methods that rely on active components, such as amplifiers and transmitters, RIS devices offer a more sustainable and cost-effective approach by leveraging passive hardware, which inherently consumes less power.

The operating principle of RIS devices is to create a virtual line-of-sight (LOS) between the signal emitter or base station (BS) and the signal receiver or user. Fig. 1 illustrates how RIS can effectively bypass obstacles and mitigate signal blockages by manipulating the EM waves, ensuring robust and reliable communication links. This capability makes RIS technology highly versatile and valuable in various scenarios. Applications of RIS cover a wide range of scenarios in modern wireless networks, including urban environments with dense building infrastructure, rural areas with limited communication facilities, and complex indoor settings like factories and offices. By strategically deploying RIS devices, it is possible to create smart radio environments that adapt to dynamic user requirements and environmental conditions, thereby overcoming the limitations of conventional communication systems. RIS technology also increases the signal-to-noise (SNR) ratio and, consequently, energy efficiency.

The advantages of RIS technology do not only prevent the signal from being blocked by obstacles, but also have the potential to ensure uninterrupted communications despite user movement by tracking the target user position [7]. Furthermore, in more complex beamforming applications, the incident signal can be divided into multiple beams and redirected toward multiple users [6,8]. Even in the field of sensing, this technology could have several applications, like Radio Frequency (RF) imaging, where high-resolution images of environment objects are generated [9], or human posture recognition systems [10]. RIS technology is currently a prominent research area, and a vast array of potential applications have been identified in recent studies.

Depending on the application and throughput requirements, different hardware configurations and operational modes are used in RIS systems. RIS structures are generally categorized into Patch Array-Based RIS and Metasurfaces. Patch Array-Based RIS create a discrete phase shift profile across the surface, leading to discontinuities [11]. Common elements used in this type of RIS include positive/intrinsic/negative (PIN) diodes, varactor diodes and delay lines, among other technologies. In contrast, Metasurfaces consist of densely arranged periodic cells, with dimensions ranging from a few millimeters to micrometers, or even to the molecular scale. These metasurfaces enable phase changes through physical modifications in the unit cell. Depending on the number of bits or states that can be achieved, they allow for a smoother manipulation of EM waves [4–6,11]. Regardless of the hardware implementation of the RIS devices, all cell elements must be controlled to modify the RIS pattern in order to satisfy real-time operational requirements of the given application. The reconfiguration time includes both the computational time required to optimize the RIS configuration and the actuation time over all the cells within the RIS device.

Due to their inherent reconfigurability, which facilitates seamless integration with electronic circuits, as well as their capability for parallel processing that supports high-speed operation, Field Programmable Gate Array (FPGA) devices are widely used in the literature for controlling cell arrays and managing the electronic circuits necessary for the independent activation or deactivation of each cell [10,12–15].

On the other hand, optimizing the phase shift in each cell of a RIS poses a distinct challenge. Various mathematical approaches for this optimization are described in the literature, many of which rely on iterative calculations [15,16] like the Gradient Descent (GD) algorithm [17,18]. Metaheuristics-based methods are also used, such as Genetic Algorithms (GA) [19], which explore a population of potential RIS configurations and iteratively improve upon them to minimize the error between the desired and current radiation patterns. If none of the configurations meet the stop criteria, defined as the minimum error between the desired and obtained radiation patterns, a new population is generated through recombination and mutation of the best RIS configurations. This process continues until a solution that satisfies the stop criteria is found. The work presented in [20] makes use of a non-dominated sorting genetic algorithm (NSGA-II) to determine the RIS configuration by optimizing the concentration of electromagnetic energy within a specific region of the surface, enabling the desired beam steering. The approach starts from an initial RIS state and repeatedly applies random modifications until the target signal amplitude is achieved. As a result, the optimization process is computationally intensive, with the generation of a single RIS configuration requiring up to three minutes on a 20-core Intel® Xeon® server, making it poorly suited for edge computing scenarios. Moreover, this problem is more complex in scenarios with a high multi-path load, such as urban and indoor situations with multiple objects, where a system configuration prioritizing a specific angle may not be the most appropriate and multibeam configurations can be required.

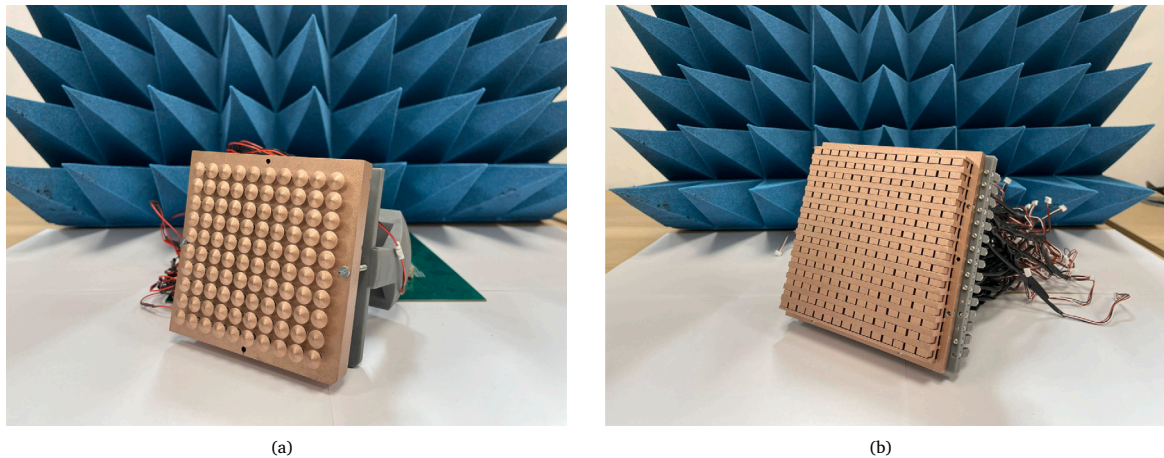
The methods above are thus often resource-intensive and time-consuming, which increases the complexity of ensuring precise and efficient phase shift adjustments for specific RIS applications. Therefore, new approaches to compute the optimal RIS configuration with innovative Artificial Intelligence (AI) algorithms have been explored [17]. Shan et al. [19] proposed a Convolutional Neural Network (CNN) that can predict the RIS configuration based on the desired beam pattern. The authors also remark the superior performance of the CNN compared to the Genetic Algorithm counterpart when running both on an Intel® Xeon® CPU. While the GA requires about 38 s to compute a configuration, the deep learning approach takes an average of 500  $\mu$ s, or 2000 configurations per seconds, on a high performance computer. Similarly, Fernández-Rodríguez et al. [21] introduced a new deep learning model specifically tailored for edge-computing, which is particularly suitable for real-time applications and resource-constrained environments. This work however only focuses on the development of the deep learning algorithm and does not address its implementation.

Taking this approach one step further, some researchers have been exploring the implementation of deep learning models on edge devices. For instance, the work presented in [22] proposes the use of FPGAs and compares their performance to other low-resource devices to accelerate inference in deep learning models for optimal RIS configuration. In this study, an image representation of the desired radiation pattern was used as the input to the NN. Results show that FPGA devices can outperform CPUs, TPUs or GPUs in DL inference tasks, achieving up to 971,14configuration per seconds. Another proposals [15,23] pre-compute the RIS configurations using computationally-intensive algorithms and create a codebook from which the appropriate configuration is selected according to the target angle. This has become a widespread strategy in RIS applications, where deep learning is used for target detection to obtain the pitch and azimuth angles of the target object relative to the RIS [15,23]. The RIS device is then configured using the pre-computed configuration that corresponds to the target's direction.

Despite the strengths of traditional CPU and GPU-based implementations, they often struggle to meet the stringent performance and energy efficiency requirements of edge computing environments. This has led to increased interest in the use of FPGAs for the application of deep learning [24,25], where their reconfigurable nature and parallel processing capabilities offer promising solutions. The main advantage of FPGAs in the edge computing paradigm is their ability to not only accelerate any specialized computation, as is the case for RIS configurations, but to also host any additional hardware required in the edge. In the case of RIS applications, a FPGA can thus integrate the accelerator for the computation of the device configuration along with all ancillary systems, from communications to control or sensing. Moreover, the availability of large numbers of configurable I/Os allows the control of devices with many elements, which would require custom serial communications and off-chip hardware if controlled by traditional CPUs or GPUs. The limit on the systems that can be integrated in a single device is only imposed by the device size of the FPGA. Reconfigurability and flexibility of FPGAs streamline system design, reduce latency even further, and open up new possibilities for more efficient, intelligent wireless communication systems.

Notably, the proposed implementation facilitates the practical deployment of RIS in challenging indoor and outdoor scenarios where no LOS link exists between the 5G/6G BS and the user equipment (UE). For instance, this type of FPGA-enabled, edge-computing RIS device can be deployed to redirect the signal from a BS located in a corridor toward a certain UE inside a room [26,27]. In outdoor scenarios, the signal from a rooftop-mounted 5G BS can be redirected toward users located below the rooftop level [27,28], using a RIS panel placed on a building facade or an utility pole to improve coverage in areas where the direct signal from a base station is blocked by buildings. Installing a RIS on a wall, ceiling or even a smart window can also enhance signal strength for high-data-rate users in office or factory environments [29]. Moreover, BSs are often deployed at the entrance of tunnels or shopping malls with long indoor pathways, where coverage rapidly degrades as users move away from the entrance; in these scenarios, RIS-assisted signal redirection can significantly improve connectivity [27]. This implementation can also enhance system-level security by steering signals toward legitimate users and reducing signal leakage to potential eavesdroppers [30–32]. Furthermore, the proposed implementation can improve radar sensing by enabling detection in non-line-of-sight (NLOS) conditions through controlled indirect propagation paths, thereby extending detection coverage into shadowed areas [33,34]. Real-world tests show that 64-QAM 5G NR signals at 24.5 GHz can achieve throughputs over 1 Gbit/s with over 20 dB improvement in signal-to-noise ratio (SNR) when using RIS to overcome blockage [35].

This paper presents a novel edge-computing implementation of the calculation of the optimal RIS configuration, using neural networks (NN) and hardware acceleration. This approach addresses a significant gap in current RIS research and enables real-time deployment, thanks to detailed tailoring and hardware-oriented fine-tuning of the CNN-based algorithm. Additionally, it provides



**Fig. 2.** Prototypes of mechanical RIS devices: (a)  $10 \times 10$  device with electromagnet actuators and (b)  $15 \times 15$  device with stepper motor actuators.

a comparison of computational costs across various RIS configurations using the proposed real-time optimization approach. It is worth noting that this approach can be extended to a variety of application fields. The remainder of this document is organized as follows: Section 2 describes the neural network to address the RIS optimization challenge and details the hardware devices and workflows used for AI acceleration. Section 3 is devoted to the details of the fine-tuning of the CNN for implementation on each hardware platform. The corresponding results are presented in Section 4, followed by a discussion in Section 5.

## 2. Materials and methods

As outlined in the previous section, the objective of this work is to accelerate the deep learning-based calculation of the optimal RIS configuration. This configuration is able to reflect the incident EM wave, making its main beam to point in the desired direction. The work presented here has been developed for two prototype mechanical RIS devices, shown in Fig. 2: a  $10 \times 10$  RIS using electromagnet actuators [36], and a  $15 \times 15$  RIS using stepper-motor actuators [37], which is still under development. In addition, a potential implementation is proposed for a future  $20 \times 20$  RIS device, illustrating the scalability of the approach. In the case of the  $15 \times 15$ , stepper-motor prototype, it includes a number of auxiliary low-cost FPGAs controlling the stepper motors and additional hardware for external communications and interfacing. With the approach presented here, a single FPGA can host all these systems along with the proposed DL-accelerator for RIS configuration, as well as any other additional systems that could be required: communications, user interfaces, additional sensing or control elements, peripherals, etc. The video in the supplementary material shows the stepper-motor RIS prototype switching between different configurations, along with the setup for RF measurements and the full control system of the RIS device.

This work presents the FPGA implementations of DL algorithms to configure the radiating element in real time. The configuration of this radiating element can produce a pointing at a certain angle, which is the case in this proposal, or a certain radiation pattern depending on more sophisticated input variables to the DL framework. Thus, the input parameters for the configuration of the radiating element depend on the application, which may be the algorithm used to maximize the SNR of users in 5G networks, a system using RIS for tracking, or inserting this type of radiating element into microwave imaging systems. All these applications will have different input parameters but the basic hardware-enabled DL structure will be very similar. Thus, the proposed approach reconfigures the radiating system in real time using a specific target angle as input. While Cross-Entropy or Markov Chain Monte Carlo can be used for multipath interference, they will not be considered here since the proposed DL focuses on learning relationships between beam angles and RIS patterns, without taking dynamic adjustments in consideration. The methodology employed to achieve this goal involves framing the optimization problem as a classification task. Specifically, for the redirection of the target beam, each RIS cell is classified into one of  $M$  classes. Each class represents the state of the RIS cell that contributes to the desired overall RIS configuration. This section provides a detailed description of the deep learning algorithm used to solve the RIS optimization problem as well as the key aspects related to the RIS nature considered in the training process.

On the other hand, in order to meet real-time operation, the proposed deep learning algorithm is accelerated taking advantage of the inherent reconfigurability of FPGA devices, which enables parallel data processing. Consequently, this section also describes two different workflows based on Altera® (formerly Intel® FPGA) and AMD platforms, reviews the unique aspects of each platform and outlines the steps involved in deploying DL algorithms on these systems, highlighting any key differences in the respective approaches to hardware acceleration.

### 2.1. Proposed neural network

The optimization problem aims to determine the optimal configuration of RIS cells such that their combined state redirects the incoming signal toward a specified target direction. The input to this problem is the target direction defined by two angles,  $\phi$  and  $\theta$  [38], representing the desired direction of the reflected signal, with  $\phi \in [25^\circ, 25^\circ]$  and  $\theta \in [-89^\circ, 90^\circ]$ . The algorithm must process these inputs to compute the optimal phase shift for each RIS element. In this study, three RIS configurations with 100, 225, and 400 reflecting elements are analyzed. Each element is capable of introducing a phase shift of either  $0^\circ$  or  $180^\circ$ . Consequently, the desired output is represented as either a  $10 \times 10$ ,  $15 \times 15$  or  $20 \times 20$  binary matrix, where each element corresponds to the state of the corresponding RIS cell.

To tackle this optimization challenge, in [21] it is proposed to frame it as a classification problem with, for example, 100 outputs corresponding to the  $10 \times 10$  RIS array cell, each with two possible classes. The DL algorithm assigns a likelihood to each cell, indicating whether it should be in the '0' or '1' state, effectively determining whether the cell induces a phase shift of  $0^\circ$  or  $180^\circ$ . The NN used is a lightweight CNN, specifically designed for edge computing. The input to this CNN is derived from the target direction angle components, namely  $\sin(\phi)$ ,  $\cos(\phi)$ ,  $\sin(\theta)$ , and  $\cos(\theta)$ . Since the input of the network is a 1D tensor, an initial fully connected layer followed by a sequence of convolutional transpose layers are applied, with the aim of expanding the dimension and extracting relevant features. Thus, the hidden tensor is a 3D-tensor and a sequence of convolutional layers can be applied for the final feature extraction. Configurations with multi-layer perceptrons (MLPs) were explored in preliminary experiments, but they were found to perform poorly for this use case, with very high error rates. Moreover, small convolutional layers were used to keep the network as small and simple as possible to simplify their hardware implementation. The network is structured as a decoder, generating ever increasing arrays, until the RIS size is reached. While it is technically possible to use fewer layers, this would have the effect of reducing the convolutional nature of the network, thus gradually increasing the size of the tensors from the beginning to the end of the network.

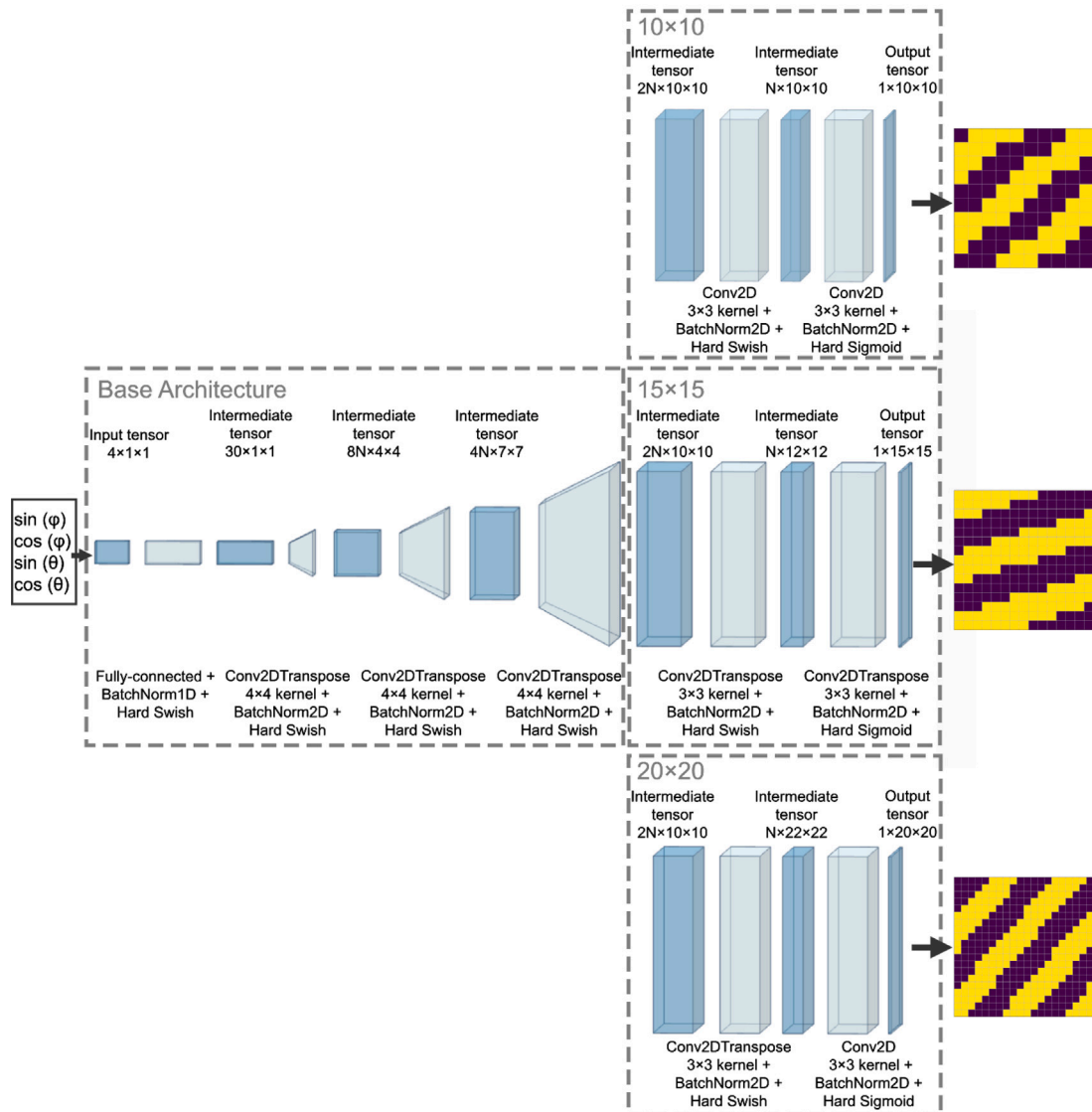
This NN is designed with a common base architecture and adapted in the final two convolutional structures to achieve the desired output dimension. The architecture features a design parameter,  $N$ , to scale the number of kernels in each convolutional layer. These configurations yield six unique model structures. For the  $10 \times 10$  RIS configuration, the network with  $N = 25$  has 509.140 parameters, while the network with  $N = 50$  leads to a significantly larger model with 1.840.540 parameters. For the  $15 \times 15$  configuration, the model sizes increase to 509.390 parameters ( $N = 25$ ) and 1.841.040 ( $N = 50$ ). Finally, for the  $20 \times 20$  RIS configuration, the networks comprise 517.965 parameters for  $N = 25$ , and 1.875.960 parameters for  $N = 50$ . The NN architectures are illustrated in Fig. 3 and detailed in Table 1.

It must be noted that architectures that use the highest value,  $N = 50$ , of the design parameter achieve significantly better accuracy than those with  $N = 25$  across all RIS configurations. For instance, in the  $10 \times 10$  configuration, accuracy increases from 92.68% with  $N = 25$  to 99.14% with  $N = 50$ . The accuracy metrics, computed from the differences between the CNN classifications and the set of ground truth [22], are summarized in Table 2. All the NNs were trained considering that there are two valid outputs for each input: the original RIS configuration and its complementary configuration. In the 1-bit RIS, each unit cell can be configured in one of two states, each corresponding to a  $180^\circ$  phase shift. If all RIS elements are shifted by  $180^\circ$ , the overall phase shift of the signal remains unchanged. Therefore, the problem is symmetrical, and a custom loss function was developed to account for this duality [21,22]. This loss function compares the NN output not only to the ground truth but also to its complementary configuration, i.e., the logical NOT of each cell, selecting the minimum of the two as the effective loss. This custom loss function ensures that the neural network appropriately accounts for the duality of the RIS configurations. Although the training dataset includes only one of these valid configurations for each input, the network is trained to recognize that the complementary configuration is equally correct. By minimizing the loss of either the predicted configuration or its complement, the network can accurately learn to produce one of the valid outputs. The proposed method effectively captures the inherent symmetry of the RIS configuration problem, resulting in a more robust training and better overall performance.

Originally, the network was trained using the SiLU (Sigmoid Linear Unit) activation function applied after both the deconvolutional and convolutional layers [21] because it was found to produce significantly better results than simpler functions such as ReLU. However, due to the non-linear nature of SiLU, it poses challenges for efficient computation, particularly when implemented on FPGA hardware. To address this issue, after a first training phase from scratch using SiLU, the networks were fine-tuned in a second training phase replacing each instance of SiLU by a linear approximation, known as the Hard-SiLU or Hard-Swish function. This change not only facilitates easier implementation on FPGAs, where linear operations are more efficiently handled, but also helps to enhance performance, improving both speed and resource efficiency.

### 2.2. Target devices

This work explores hardware acceleration using two different FPGA prototyping platforms: the Altera® Arria® 10 SX SoC Development Kit [39] and the AMD Versal™ VCK-190 Evaluation Kit [40]. The Altera® Arria® 10 is a System-on-Chip (SoC) that combines an Arm® processor with FPGA programmable logic, being a flexible platform for AI inference tasks. This integration establishes a clear allocation of computational tasks, whereby the Arm® processor is tasked with managing control and preprocessing operations, while the FPGA handles computation-intensive tasks. On the other hand, the AMD Versal™ VCK-190 Evaluation Kit [40] features the Versal™ XCVC1902 chip, a part of the next generation of Adaptive Compute Acceleration Platforms (ACAPs). This heterogeneous SoC integrates Adaptable Intelligence Engines (AIEs) [41], FPGA logic and scalar processors (CPUs). This makes



**Fig. 3.** NN architecture representation. Light-blue blocks represent the neural network (NN) layers, while dark-blue blocks illustrate the feature shapes. Three different alternatives are used to achieve the three distinct RIS configurations. The NN begins with the Base Architecture and, depending on the target RIS configuration, the final structure is adapted to the  $10 \times 10$ ,  $15 \times 15$  or  $20 \times 20$  sizes.

it particularly well-suited for complex, resource-intensive deep learning applications, such as real-time RIS optimization, where high throughput and low latency are essential.

While the Altera® Arria® 10 SX SoC Development Kit is clearly less powerful than the AMD Versal™ VCK-190 Evaluation Kit, it has been included in this work because this device and its associated toolchain offer floating-point (FP) implementations. On the other hand, AI implementations in the AMD Versal™ VCK-190 Evaluation Kit process 8-bit fixed-point data, which requires some careful tailoring and training to not jeopardize accuracy. Further details on each device and its corresponding implementation are provided in the supplementary material.

### 2.3. Intel® / Altera® workflow

The Intel® FPGA AI Suite [42] exploits the OpenVINO™ [43] toolkit to streamline the deployment of AI inference models on Altera® FPGAs. This integrated strategy allows for efficient conversion and optimization of pretrained models, making them suitable for FPGA implementation.

The pretrained TensorFlow/Keras model is first processed through OpenVINO™, where it is converted into an Intermediate Representation (IR) format [44]. The IR model encapsulates the entire neural network structure, including its layers, connections,

**Table 1**  
Architectural details of NNs for RIS configuration.

|                   | Layer               | $N = 25$      |                | $N = 50$      |                  |
|-------------------|---------------------|---------------|----------------|---------------|------------------|
|                   |                     | Output shape  | Parameters     | Output shape  | Parameters       |
| Base Architecture | Input               | [4, 1, 1]     | –              | [4, 1, 1]     | –                |
|                   | Fully Connected     | [30, 1, 1]    | 120            | [30, 1, 1]    | 120              |
|                   | Batch Normalization | [30, 1, 1]    | 120            | [30, 1, 1]    | 120              |
|                   | Hard Swish          | [30, 1, 1]    | –              | [30, 1, 1]    | –                |
|                   | Conv2DTranspose     | [8×N, 4, 4]   | 96.000         | [8×N, 4, 4]   | 192.000          |
|                   | Batch Normalization | [8×N, 4, 4]   | 800            | [8×N, 4, 4]   | 1600             |
|                   | Hard Swish          | [8×N, 4, 4]   | –              | [8×N, 4, 4]   | –                |
|                   | Conv2DTranspose     | [4×N, 7, 7]   | 320.000        | [4×N, 7, 7]   | 1.280.000        |
|                   | Batch Normalization | [4×N, 7, 7]   | 400            | [4×N, 7, 7]   | 800              |
|                   | Hard Swish          | [4×N, 7, 7]   | –              | [4×N, 7, 7]   | –                |
|                   | Conv2DTranspose     | [2×N, 10, 10] | 80.000         | [2×N, 10, 10] | 320.000          |
|                   | Batch Normalization | [2×N, 10, 10] | 200            | [2×N, 10, 10] | 400              |
|                   | Hard Swish          | [2×N, 10, 10] | –              | [2×N, 10, 10] | –                |
| $10 \times 10$    | Padding             | [2×N, 12, 12] | –              | [2×N, 12, 12] | –                |
|                   | Conv2D              | [N, 10, 10]   | 11.275         | [N, 10, 10]   | 45.050           |
|                   | Hard Swish          | [N, 10, 10]   | –              | [N, 10, 10]   | –                |
|                   | Padding             | [N, 12, 12]   | –              | [N, 12, 12]   | –                |
|                   | Conv2D              | [1, 10, 10]   | 225            | [1, 10, 10]   | 450              |
|                   | Hard Sigmoid        | [1, 10, 10]   | –              | [1, 10, 10]   | –                |
|                   | <b>Total:</b>       |               | <b>509.140</b> |               | <b>1.840.540</b> |
| $15 \times 15$    | Conv2DTranspose     | [N, 12, 12]   | 11.250         | [N, 12, 12]   | 45.000           |
|                   | Batch Normalization | [N, 12, 12]   | 100            | [N, 12, 12]   | 200              |
|                   | Hard Swish          | [N, 12, 12]   | –              | [N, 12, 12]   | –                |
|                   | Conv2DTranspose     | [1, 15, 15]   | 400            | [1, 15, 15]   | 800              |
|                   | Hard Sigmoid        | [1, 15, 15]   | –              | [1, 15, 15]   | –                |
|                   | <b>Total:</b>       |               | <b>509.390</b> |               | <b>1.841.040</b> |
| $20 \times 20$    | Conv2DTranspose     | [N, 22, 22]   | 20.000         | [N, 22, 22]   | 80.000           |
|                   | Batch Normalization | [N, 22, 22]   | 100            | [N, 22, 22]   | 200              |
|                   | Hard Swish          | [N, 22, 22]   | –              | [N, 22, 22]   | –                |
|                   | Conv2D              | [1, 20, 20]   | 225            | [1, 20, 20]   | 450              |
|                   | Hard Sigmoid        | [1, 20, 20]   | –              | [1, 20, 20]   | –                |
|                   | <b>Total:</b>       |               | <b>517.965</b> |               | <b>1.875.690</b> |

**Table 2**  
Accuracy of trained NNs.

| RIS configuration | $N = 25$ | $N = 50$ |
|-------------------|----------|----------|
| $10 \times 10$    | 92,68%   | 99,14%   |
| $15 \times 15$    | 88,64%   | 95,19%   |
| $20 \times 20$    | 86,85%   | 91,73%   |

weights, and biases. The accuracy results for the IR models of the different NN architectures match with those in Table 2, since these are still software-derived results.

In this case, the IR model is further processed by the DLA (Deep Learning Accelerator) Graph Compiler in the Intel® FPGA AI Suite, which tailors the model for execution on the Altera® Arria® 10 FPGA. It generates information about the layers, their connections, weights, biases, and instructions specific to the CoreDLA IP core [45]. The CoreDLA IP is built specifically for the target Altera® FPGA device based on the configuration parameters defined in the IP architecture file (.arch). The compilation and synthesis is carried out using the Quartus® Prime software [46]. This IP core can be customized using different sets of performance and area parameters, thereby offering flexibility in the allocation of resources. Once the core IP is synthesized, the generated bitstream is integrated into the Yocto Linux system, alongside the FPGA AI Suite Runtime Plugin. This setup ensures that the FPGA is fully prepared to efficiently perform AI inference.

#### 2.4. AMD workflow

The AMD workflow closely mirrors the one outlined in Section 2.3, with the primary difference laying on the fact that, in this scenario, the AI accelerator processes 8-bit fixed-point data. Therefore, the model must be quantized beforehand, which can be accomplished using either Post-Training Quantization (PTQ) or Quantization-Aware Training (QAT) techniques. In this study, the TensorFlow variant of the Vitis™ AI AI (VAI) Quantizer is used to convert the software model, originally represented in floating-point format, into an 8-bit fixed-point model. Additionally, operations not supported by the target hardware, such as Batch Normalization layers, are folded into the preceding layer during the conversion process.

**Table 3**  
Parameters of the Intel® FPGA AI Suite IP architectures for RIS accelerator evaluation.

|                                 | Precision | KVEC | Output writer<br>KVEC | CVEC | Stream buffer<br>depth |
|---------------------------------|-----------|------|-----------------------|------|------------------------|
| A10 Architecture minArea        | FP16      | 4    | 4                     | 4    | 28.238                 |
| A10 Architecture maxPerformance | FP16      | 64   | 32                    | 32   | 2978                   |
| A10 Architecture Balance        | FP16      | 16   | 16                    | 8    | 28.238                 |

The AMD implementation relies on the Deep Learning Processor Unit (DPU) cores [47]. In particular, the DPUCVDX8G for AMD Versal™ ACAP devices has been used in this work. This IP uses both the FPGA programmable logic (PL) and the AIEs, and the amount of resources in each domain can be configured.

The AIEs are the hardware resources in which the convolution operations are performed, and they are organized in groups. Each batch handler has a private group of AIEs and a memory tile group. The PL component comprises a high-level scheduler module, global memory for shared weights, and batch handlers responsible for Load, Save, Pool, and Elt-wise operations. The scheduler and weights buffers are shared across all batch handlers. Each batch handler has its own private resources, including the Load and Save Module, Pooling, and Elt-wise Modules, as well as a local feature map storage.

The VAI compiler translates the converted network model into a highly optimized sequence of DPU instructions. The process begins by parsing the topology of the optimized and quantized input model, followed by the construction of an internal computation graph as an intermediate representation (IR), which includes the corresponding control flow and data flow representations. The compiler then performs a series of optimizations, such as efficient instruction scheduling, that leverage inherent parallelism and enhance data reuse. The result is a .xmodel file, which is a serialized version of the model in XIR-based format.

Finally, the compiled deep learning model is deployed in the DPU through the Vitis AI Runtime (VART). VART is a software library that acts as the interface between the compiled model and the hardware. It provides APIs for loading the model, handling input and output data, and launching inference, while abstracting hardware-specific details to ensure efficient execution on the DPU. It provides a set of APIs that manage tasks, such as model loading, task creation, input/output data handling, and execution of inference on the DPU. It abstracts the complexities of interacting with the underlying hardware, allowing efficient deployment. This runtime environment ensures that the model runs efficiently on the DPU, taking full advantage of the hardware capabilities for parallel processing and optimized performance.

### 3. Implementation details

This section provides an in-depth description of both Altera® and AMD implementations. It highlights the architectural configurations, parameter choices, and resource optimization strategies used to tailor the accelerators for RIS applications.

#### 3.1. Altera®

As outlined in Section 2.3, the implementation on the Altera® FPGA is based on the Intel® FPGA AI Suite IP. It supports both FP11 (1s.6m.5e) and FP16 (1s.11m.5e) precision on the Altera® Arria® 10 platform. In order to maximize performance, the FP11 representation was initially considered to implement the application accelerator. However, due to the characteristics of the NN under study, this implementation struggled accuracy-wise when FP11 was used. Consequently, the final implementation adopted FP16 precision as a balance between accuracy and performance. Using this setup, three distinct accelerator architectures were evaluated, with their key parameters outlined in Table 3. These parameters have a direct effect on the performance and resource utilization of the accelerators. The Intel® FPGA AI Suite IP is based on a Processing Element (PE) Array, where the number of PEs is determined by the KVEC parameter, describing the number of filters the PE Array can process simultaneously, and the CVEC parameter, which defines the size of the dot product performed within each PE. Variations of these parameters bring forth three different architectures with different area and performance characteristics. Table 4 shows the area usage for each implementation in terms of logic elements and memory blocks.

By setting KVEC and CVEC to minimal values (both set to 4), low resource usage is prioritized in detriment of performance. The resulting 'A10 Architecture minArea' produces a minimal accelerator that enables the deployment of up to five parallel accelerators. This provides a significant advantage, not only in allowing to control five independent RIS systems with a single device hosting the five corresponding accelerators ('A10 Architecture minArea (x5)'), but also in allowing a larger RIS to be formed with five smaller RIS units. In the context of this study, this would correspond to a RIS with up to 2000 array cell elements, configured as a 5-element set of 20 × 20-cell RIS. Furthermore, a significant portion of resources would remain available for additional implementations, with around a third of the ALMs and BlockRAM memory bits and most of the Variable Precision DSPs still unused.

On the other hand, 'A10 Architecture maxPerformance' is optimized for performance. It utilizes the largest KVEC (64) and CVEC (32) values among the three architectures, allowing the accelerator to process a significantly greater number of filters and dot products simultaneously. This results in higher throughput but at the cost of increased area usage. To somehow relieve this demand of resources, the depth of the stream buffer, which is used as the on-chip cache for feature data, was optimized in this architecture.

**Table 4**

Resource usage in the Altera® Arria® 10 SX SoC Development Kit with the Intel® FPGA AI Suite.

|                                 |                | ALM     | BlockRAM<br>memory<br>bits | BlockRAM | Variable<br>Precision<br>DSP |
|---------------------------------|----------------|---------|----------------------------|----------|------------------------------|
| Altera® Arria® 10 SX 660        | <b>Total</b>   | 251.680 | 43.642.880                 | 2131     | 1687                         |
| A10 Architecture minArea        | <b>Used</b>    | 35.684  | 5.904.800                  | 360      | 28                           |
|                                 | <b>Usage %</b> | 14,18%  | 13,53%                     | 16,89%   | 1,66%                        |
| A10 Architecture minArea (×5)   | <b>Used</b>    | 178.420 | 29.524.000                 | 1800     | 140                          |
|                                 | <b>Usage %</b> | 70,89%  | 67,65%                     | 84,47%   | 8,30%                        |
| A10 Architecture maxPerformance | <b>Used</b>    | 84.304  | 19.589.408                 | 1104     | 1118                         |
|                                 | <b>Usage %</b> | 33,50%  | 44,89%                     | 51,81%   | 66,27%                       |
| A10 Architecture Balance        | <b>Used</b>    | 47.005  | 11.995.424                 | 661      | 166                          |
|                                 | <b>Usage %</b> | 18,68%  | 27,49%                     | 31,02%   | 9,84%                        |

**Table 5**

Parameters of the AMD DPU IP architectures for RIS accelerator evaluation.

|                                 | Precision | <i>B</i> | <i>CPB</i> |
|---------------------------------|-----------|----------|------------|
| DPU Architecture minArea        | INT8      | 1        | 32         |
| DPU Architecture maxPerformance | INT8      | 5        | 64         |
| DPU Architecture Balanced       | INT8      | 3        | 32         |

Finally, the ‘A10 Architecture Balance’ takes a balanced approach, with  $KVEC = 16$  and  $CVEC = 8$ . This configuration represents a trade-off between resource usage and performance, making it suitable for applications that require a mix of efficiency and computational power.

It is important to note that the resource usage includes not only the CoreDLA IP but also all necessary peripherals, including a Nios® V [48] instance, which facilitates communications between the CoreDLA IP and the HPS (Hard Processor System) CPU of the Altera® Arria® 10 SoC device. All the architectures run the CoreDLA IP at 100 MHz. Performance results for all test platforms will be discussed in Section 4.

### 3.2. AMD

Unlike the implementation on the Altera® FPGA, the architecture available for the AMD platform through Vitis™ AI only supports INT8 precision. Therefore, for this platform, the implementation has focused on achieving results similar to those obtained with floating-point in the software model, using the different quantization approaches commented in Section 2.4

The PTQ approach was initially tested, as it generally results in minimal accuracy loss. However, in this specific application, PTQ led to unacceptable misclassification of most RIS cells after quantization. This could be due to the high sensitivity of the network to the reduced precision of weights and activations, which caused significant deviations in the learned feature representations. Additionally, the distribution of activation values in certain layers exhibited heavy-tailed behavior, which the uniform quantization scheme used in PTQ could not adequately capture. These factors disrupted the decision boundaries learned during training, leading to severe degradation in the model’s ability to distinguish between RIS cell classes accurately.

Consequently, the focus was shifted to the QAT approach. In QAT, the training process is repeated taking into account that the final precision of the weights will be lower. Although the weights are kept in floating-point representation during training, they are quantized after each gradient update [49]. Additionally, the initial weights are not random but are derived from the pre-trained model, ensuring a more stable starting point. For the quantization strategy, a power-of-two scale quantizer with training quantization threshold (TQT) was used [50].

Furthermore, one more change was necessary to achieve an accuracy similar to the original NN. Although the initial model includes the Hard-Swish activation function to facilitate hardware acceleration, as detailed in Section 2.1, retraining achieved better accuracy metrics when the SiLU activation function was used instead. After QAT, the SiLU activation layer was replaced back by the Hard-Swish layer with the VAI Quantizer tool to enable the mentioned hardware acceleration. The retraining process was carried out for 2000 epochs to closely recover the initial accuracy of the various models. Furthermore, the number of epochs was kept consistent across all retraining procedures to ensure a fair comparison of the results among the different models.

Similarly to what was discussed for the Altera® platform, the AMD DPU IP has two different configuration parameters that have a direct impact on performance and resource utilization. The first parameter configures the number of AIE cores used per batch (*CPB*). The second parameter configures the number of batches (*B*) that can be processed at the same time, which will determine the number of batch handlers implemented in the FPGA PL.

By adjusting the *B* and *CPB* parameters, the amd DPU IP can be tailored into three specific configurations listed in Table 5, each optimized for different priorities such as resource efficiency or performance. The first configuration, referred to as the ‘DPU Architecture minArea’, sets  $B = 1$  and  $CPB = 32$ . This design prioritizes minimal resource usage, enabling the deployment of

**Table 6**

Resource usage in the AMD Versal™ VCK-190 Evaluation Kit with the AMD DPU.

|                                 |                | LUT     | UltraRAM<br>memory bits | UltraRAM<br>block | DSP58  | AIE |
|---------------------------------|----------------|---------|-------------------------|-------------------|--------|-----|
| Versal™XCVC1902                 | <b>Total</b>   | 899.840 | 136.544.256             | 463               | 1968   | 400 |
| DPU Architecture minArea        | <b>Used</b>    | 82.942  | 35.127.296              | 136               | 139    | 32  |
|                                 | <b>Usage %</b> | 9,22%   | 29,37%                  | 29,37%            | 7,06%  | 8%  |
| DPU Architecture minArea (×3)   | <b>Used</b>    | 248.826 | 105.381.888             | 408               | 417    | 96  |
|                                 | <b>Usage %</b> | 27,65%  | 88,12%                  | 88,12%            | 21,19% | 24% |
| DPU Architecture maxPerformance | <b>Used</b>    | 355.909 | 102.236.160             | 392               | 675    | 320 |
|                                 | <b>Usage %</b> | 39,55%  | 74,87%                  | 84,67%            | 34,3%  | 80% |
| DPU Architecture Balance        | <b>Used</b>    | 209.657 | 68.681.728              | 264               | 407    | 96  |
|                                 | <b>Usage %</b> | 23,3%   | 57,02%                  | 57,02%            | 20,68% | 24% |

multiple parallel accelerators, up to three as shown in Table 6 ('DPU Architecture minArea (×3)'), although at the expense of reduced throughput and processing speed.

The 'DPU Architecture Balanced' configuration increases the batch size to  $B = 3$  while maintaining  $CPB = 32$ . This approach strikes a balance between resource usage and performance, achieving a moderate improvement in throughput without excessively increasing hardware requirements. It is particularly well-suited for scenarios where a balance between area constraints and performance targets is desired.

Finally, the 'DPU Architecture maxPerformance' configuration maximizes performance by setting  $B = 5$  and  $CPB = 64$ . This implementation fully leverages the available hardware resources to deliver the highest possible throughput and computational speed, at the cost of significantly increasing the resource utilization. These three configurations allowed to explore a wide design space and tailor the DPU to meet the specific requirements and constraints of this application. Table 6 summarizes the resource utilization for each implementation in terms of logic elements, memory blocks, DSP embedded multipliers, and AIEs.

## 4. Results

This section provides an overview of the results obtained after deploying the neural network models on the two FPGA platforms evaluated in this study: the Altera® Arria® 10 and the AMD Versal™. The following subsections will present a comprehensive analysis of how each platform performed in terms of the accuracy of the inferences and the performance metrics for each implementation.

### 4.1. Accuracy

The accuracy comparing the ground truth RIS configuration and the output from the FPGA inference has been evaluated as the overall system accuracy. It is worth noting that the accuracy evaluation must take into account the complementary nature of the RIS configurations. For a 1-bit RIS, both the labeled configuration and its logical complement can redirect the signal to the desired direction with equal effectiveness, as discussed in Section 2.1. Therefore, the inferred output is compared against both the labeled configuration and its logical complement, and the error is calculated based on the minimal difference between the two. The error is computed based on the cell-by-cell difference between the predicted and expected configurations. The hit rate is then averaged across all predictions for each accelerator under study. Table 7 provides a summary of the accuracy results for the different platforms.

For the Altera® Arria® 10, a slight reduction in accuracy was observed due to the transition from a FP32-based software model to FP16 implementations, which is necessary for efficient deployment on this platform, as discussed in Section 3. This error derived from the hardware implementation was assessed by comparing the results inferred by the different FPGA implementations with the outputs generated by the software model, with this comparison summarized in Table 8. In the case of the Altera® Arria® 10 implementation, the hardware implementation scores an average 98,34% match with the software-predicted results, this difference accounting for that decrease in accuracy from the ground truth.

Similarly, on the AMD Versal™, quantization to INT8 led to an additional slight decrease in accuracy. Nevertheless, this decrease remained within a 4% range relative to the accuracy of the original models, which is acceptable given the significantly reduced precision of the weights. In the same way that was analyzed for the Altera® Arria® 10 implementations, Table 8 also compares the results from the AMD Versal™ hardware implementations to the original software models. Thus, the discrepancies are greater in this case when compared to those observed with the Altera® Arria® 10. This is caused by the need to retrain all models as part of the quantization process; in this way, the AMD implementations fail to predict different cells to those failed by the software model when both predictions are compared to the ground truth. Consequently, while the accuracy relative to the ground truth remains similar for both Altera® Arria® 10 and AMD Versal™ devices, the predictions from the two implementations exhibit more pronounced differences when directly compared. The Altera® Arria® 10 results are closer to the software model thanks to the use of floating-point precision and the absence of any retraining. However, both platforms have demonstrated robust performance in real-time RIS optimization tasks, confirming the effectiveness of the hardware acceleration strategies described in this work.

**Table 7**

Comparison of NN system overall accuracy (vs ground truth) across the different devices and implementations.

| Device            | Architecture          | $10 \times 10$ |          | $15 \times 15$ |          | $20 \times 20$ |          |
|-------------------|-----------------------|----------------|----------|----------------|----------|----------------|----------|
|                   |                       | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ |
| Altera® Arria® 10 | <b>minArea</b>        | 92,19%         | 98,75%   | 88,03%         | 94,49%   | 86,01%         | 91,01%   |
|                   | <b>maxPerformance</b> | 92,19%         | 98,74%   | 88,03%         | 94,49%   | 86,01%         | 91,00%   |
|                   | <b>Balance</b>        | 92,19%         | 98,75%   | 88,04%         | 94,49%   | 86,01%         | 91,01%   |
| AMD Versal™       | <b>minArea</b>        | 90,71%         | 95,02%   | 86,57%         | 93,36%   | 85,48%         | 90,94%   |
|                   | <b>maxPerformance</b> | 90,71%         | 95,02%   | 86,57%         | 93,36%   | 85,48%         | 90,94%   |
|                   | <b>Balance</b>        | 90,71%         | 95,02%   | 86,57%         | 93,36%   | 85,48%         | 90,94%   |
| Software models   |                       | 92,68%         | 99,14%   | 88,64%         | 95,19%   | 86,85%         | 91,73%   |

**Table 8**

Comparison of NN hardware implementation accuracy (vs software results) across the different devices and implementations.

| Device            | Architecture          | $10 \times 10$ |          | $15 \times 15$ |          | $20 \times 20$ |          |
|-------------------|-----------------------|----------------|----------|----------------|----------|----------------|----------|
|                   |                       | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ |
| Altera® Arria® 10 | <b>minArea</b>        | 98,05%         | 99,32%   | 98,45%         | 98,33%   | 97,96%         | 97,98%   |
|                   | <b>maxPerformance</b> | 98,00%         | 99,31%   | 98,45%         | 98,33%   | 97,93%         | 97,96%   |
|                   | <b>Balance</b>        | 98,02%         | 99,31%   | 98,45%         | 98,32%   | 97,94%         | 97,97%   |
| AMD Versal™       | <b>minArea</b>        | 89,98%         | 94,45%   | 88,48%         | 92,90%   | 88,69%         | 89,84%   |
|                   | <b>maxPerformance</b> | 89,98%         | 94,45%   | 88,48%         | 92,90%   | 88,69%         | 89,84%   |
|                   | <b>Balance</b>        | 89,98%         | 94,45%   | 88,48%         | 92,90%   | 88,69%         | 89,84%   |

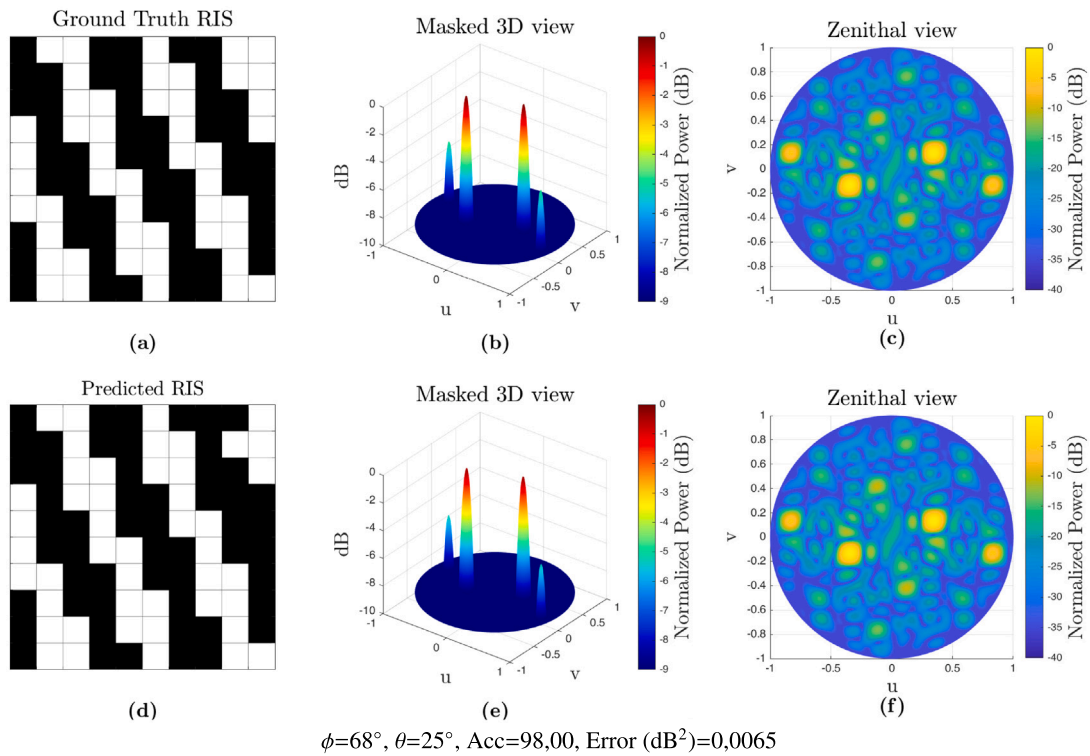
**Table 9**Comparison of NN hardware implementation radiation error (vs ground truth RIS configuration) across the different devices and implementations expressed in dB<sup>2</sup>.

| Device            | Architecture          | $10 \times 10$ |          | $15 \times 15$ |          | $20 \times 20$ |          |
|-------------------|-----------------------|----------------|----------|----------------|----------|----------------|----------|
|                   |                       | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ |
| Altera® Arria® 10 | <b>minArea</b>        | 4,17E-2        | 7,00E-3  | 1,15E-2        | 1,05E-2  | 5,00E-3        | 5,20E-3  |
|                   | <b>maxPerformance</b> | 4,15E-2        | 7,10E-3  | 1,15E-2        | 1,04E-2  | 5,00E-3        | 5,20E-3  |
|                   | <b>Balance</b>        | 4,18E-2        | 7,00E-3  | 1,15E-2        | 1,03E-2  | 5,00E-3        | 5,20E-3  |
| AMD Versal™       | <b>minArea</b>        | 3,70E-2        | 3,48E-2  | 1,91E-2        | 1,07E-2  | 1,30E-2        | 7,20E-3  |
|                   | <b>maxPerformance</b> | 3,70E-2        | 3,48E-2  | 1,91E-2        | 1,07E-2  | 1,30E-2        | 7,20E-3  |
|                   | <b>Balance</b>        | 3,70E-2        | 3,48E-2  | 1,91E-2        | 1,07E-2  | 1,30E-2        | 7,20E-3  |
| Software models   |                       | 3,61E-2        | 4,70E-3  | 1,40E-2        | 9,20E-3  | 9,10E-3        | 7,00E-3  |

Overall system precision was also evaluated through the electromagnetic radiation pattern generated by each RIS configuration. For every hardware implementation, the radiation profile corresponding to each inferred RIS configuration is computed. The error is then calculated as the point-wise difference in radiated power, over a predefined mesh, between the pattern produced by the hardware-accelerated RIS configuration and the expected pattern from the ground-truth configuration. To focus only on the main beam error and disregard secondary lobes that are not relevant for signal transmission, a  $-9$  dB mask is applied to both the expected and the produced radiation patterns. Consequently, the error is computed as the difference between expected and produced power at each mesh point, averaged over all points where the mask applies. The error is expressed in dB<sup>2</sup>, which reflects the squared difference of two power levels in decibels and, therefore, quantifies the energy of the deviation rather than just its absolute amplitude. The same procedure is applied to the RIS configurations obtained from the NN software models to enable a direct comparison with the hardware implementation. The mean values across all resulting RIS configurations are summarized in Table 9. Fig. 4 illustrates the  $10 \times 10$ -RIS configuration corresponding to the  $\phi = 68^\circ$ ,  $\theta = 25^\circ$  target redirection angle, showing both the ground-truth configuration and the predicted configuration. Next to them, the radiation patterns of both RIS configurations are displayed: first, a  $-9$  dB mask has been applied to the 3D-view of the radiation patterns, as discussed above, so only the main beams remain visible. On the left of these 3D views, zenithal views of the full U-V representations depict the complete radiation patterns without any masking, so all beams and side lobes are displayed. For this particular example in Fig. 4, accuracy referred to RIS-cell predictions is 98,00%, while the resulting radiation error is 0,0068 dB<sup>2</sup>. Figs. 5 and 6 show the corresponding representation for the same target redirection angle for the  $15 \times 15$  and  $20 \times 20$  RIS devices, respectively. The achieved cell-level accuracies are 99,11% and 92,00%, respectively, while the corresponding radiation errors are 0,0007 dB<sup>2</sup> and 0,0003 dB<sup>2</sup>.

#### 4.2. Performance

The performance of the RIS accelerators have been evaluated in terms of the number of RIS configurations per second (CPS), which serves as the key indicator of system throughput. This metric has been computed as the inverse of the inference time that



**Fig. 4.** Ground-truth and predicted  $10 \times 10$  RIS configurations for the  $\phi = 68^\circ, \theta = 25^\circ$  target angle alongside their corresponding radiation patterns. (a) and (d) show the ground-truth and the predicted RIS configurations (b) and (e) display 3D-views of the radiation patterns when a  $-9$  dB mask is applied. (c) and (f) present zenithal views of the complete radiation patterns.

it takes to compute a single RIS configuration, which has been measured and averaged across all the input angles in the entire dataset. The results represent the maximum number of configurations achievable by each accelerator regardless of the hardware used to actuate the RIS device. The detailed values are presented in Table 10.

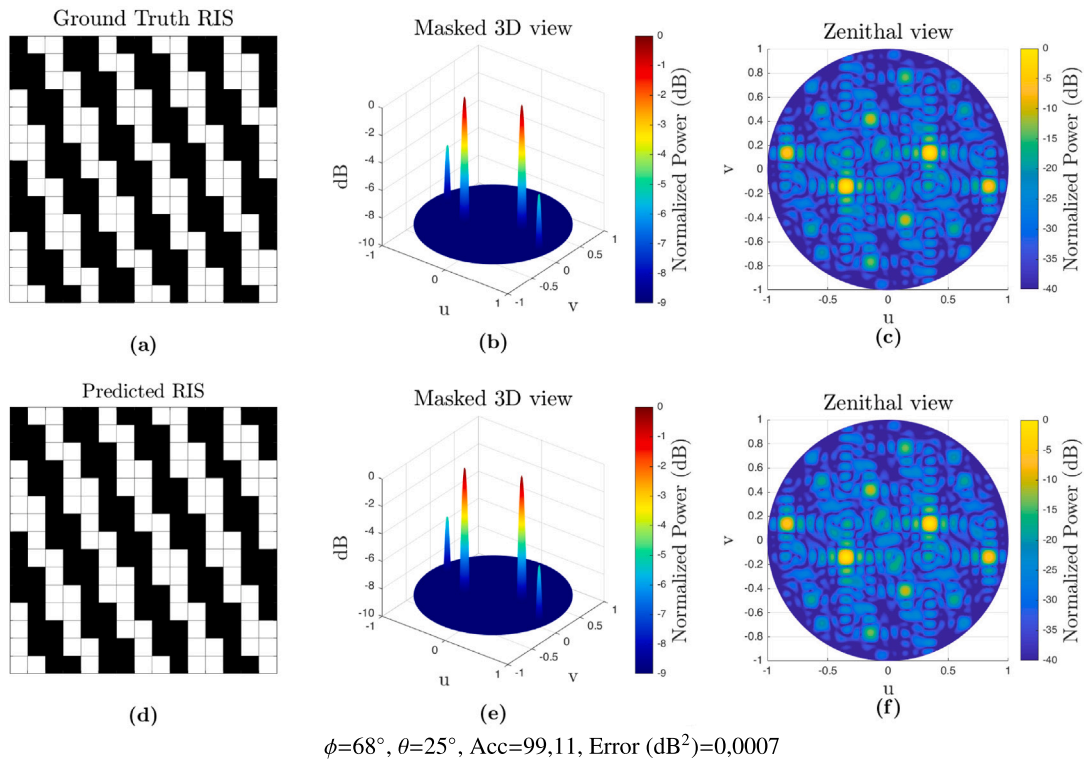
The Altera® Arria® 10, utilizing FP16 precision and relying solely on its FPGA resources, achieves a computational efficiency that is well-suited for applications requiring moderate throughput and precision. Its performance demonstrates a balance between resource utilization and output quality, making it suitable for less demanding workloads where better precision at a slightly higher latency is acceptable. However, its traditional FPGA architecture is better suited for tasks that are less computationally intensive.

In contrast, the AMD Versal™ uses INT8 quantization, a lower-precision format that significantly accelerates computation while maintaining acceptable accuracy levels for most AI workloads. Beyond the benefits of INT8 quantization, the AMD Versal™ leverages its heterogeneous architecture, which combines classical FPGA resources with dedicated AI Engines. These AI Engines are specifically optimized for matrix multiplication operations, enabling highly parallelized data processing and faster inference times. This dual-architecture approach not only improves latency and throughput but also allows the AMD Versal™ to efficiently handle complex, resource-intensive tasks that require high computational power.

#### 4.3. Resource usage and performance analysis

The resource usage for both FPGA platforms is assessed to understand how effectively each system utilizes its available hardware. The Altera® Arria® 10 relies entirely on DSPs to perform computationally intensive tasks such as matrix multiplications and convolutions. Consequently, the performance efficiency of this device can be analyzed through the CPS/DSP metric presented in Table 11. This metric represents the number of RIS configurations processed per second, normalized by the number of dedicated resources employed, offering a clear view of how well the system maximizes its resources. In the case of the Altera® Arria® 10, these resources are limited to the Variable Precision DSP multipliers, while the AMD Versal™ includes both DSP58s and AIEs.

The Altera® Arria® 10 architectures analyzed in this work, ‘A10 Architecture minArea’ and ‘A10 Architecture maxPerformance’ exhibit similar CPS/DSP ratios, indicating comparable efficiency in processing neural network models despite their differing design priorities. In contrast, the ‘A10 Architecture Balance’ stands out by achieving a higher performance efficiency compared to the other two architectures. This increased efficiency suggests that this last architecture strikes a more optimal balance between resource usage and computational throughput.



**Fig. 5.** Ground-truth and predicted  $15 \times 15$  RIS configurations for the  $\phi = 68^\circ, \theta = 25^\circ$  target angle alongside their corresponding radiation patterns. (a) and (d) show the ground-truth and the predicted RIS configurations (b) and (e) display 3D-views of the radiation patterns when a  $-9$  dB mask is applied. (c) and (f) present zenithal views of the complete radiation patterns.

**Table 10**

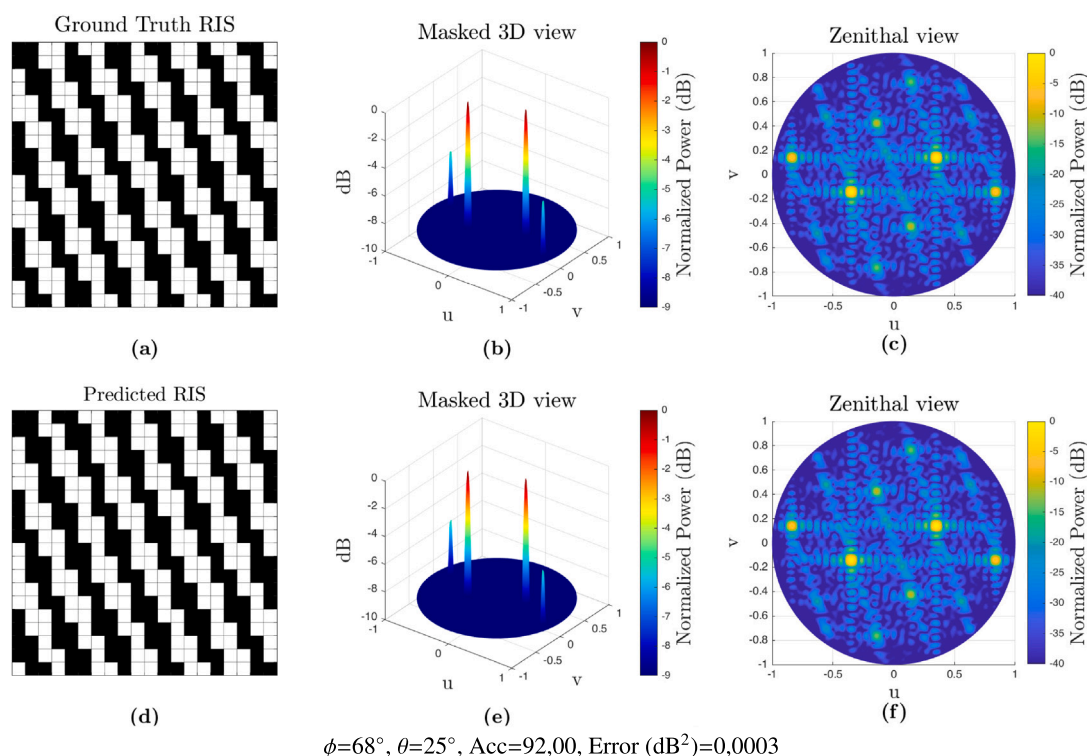
CPS performance of NN execution across the different devices and implementations.

| Device            | Architecture          | $10 \times 10$ |           | $15 \times 15$ |           | $20 \times 20$ |           |
|-------------------|-----------------------|----------------|-----------|----------------|-----------|----------------|-----------|
|                   |                       | $N = 25$       | $N = 50$  | $N = 25$       | $N = 50$  | $N = 25$       | $N = 50$  |
| Altera® Arria® 10 | <b>minArea</b>        | 110,45         | 29,61     | 106,83         | 28,83     | 101,20         | 27,44     |
|                   | <b>maxPerformance</b> | 4520,11        | 1509,98   | 4096,59        | 1422,24   | 3791,36        | 1362,83   |
|                   | <b>Balance</b>        | 1294,43        | 397,43    | 1218,36        | 378,96    | 1144,53        | 355,532   |
| AMD Versal™       | <b>minArea</b>        | 5469,28        | 3861,12   | 5146,03        | 3825,70   | 4796,03        | 3367,88   |
|                   | <b>maxPerformance</b> | 18.390,41      | 15.253,45 | 18.015,21      | 14.972,58 | 16.888,06      | 13.598,69 |
|                   | <b>Balance</b>        | 12.834,23      | 9988,93   | 12.691,13      | 9868,65   | 11.443,48      | 8697,41   |

On the other hand, the AMD Versal™ employs a hybrid architecture that combines DSP slices with AIEs. This integration allows the device to achieve far greater performance by offloading matrix-heavy operations, such as those found in deep learning inference, to its highly efficient AIEs. While the CPS/DSP metric remains relevant for comparing the performance of its DSP resources alone, the inclusion of AIEs introduces a new metric, CPS/(DSP+AIE), which reflects the combined efficiency of both resource types. Table 12 shows the results of this metric for the three architectures discussed in this manuscript. The ‘DPU Architecture minArea’ demonstrates the highest average CPS/(DSP+AIE) figure. In contrast, the ‘DPU Architecture maxPerformance’ prioritizes throughput, resulting in higher overall CPS but a noticeable reduction in average CPS. This reflects the trade-off between maximizing raw computational power and achieving efficient utilization of the available resources. The ‘DPU Architecture balanced’ provides a compromise between these extremes. While the AMD Versal™ demonstrates higher CPS/(DSP+AIE) for the ‘DPU Architecture minArea’ than for the ‘DPU Architecture balanced’, the Altera® Arria® 10 improved CPS/DSP for the balanced configuration suggests that its architecture benefits more from a moderate increase in resource allocation without overloading its DSPs, showcasing the differing ways these architectures respond to resource scaling.

## 5. Conclusions

In this research work, a deep learning approach for optimal RIS configuration across two distinct FPGA devices has been evaluated and implemented. Thus, a custom CNN for RIS configuration has been tailored for hardware implementation on the edge. An example



**Fig. 6.** Ground-truth and predicted  $20 \times 20$  RIS configurations for the  $\phi = 68^\circ, \theta = 25^\circ$  target angle alongside their corresponding radiation patterns. (a) and (d) show the ground-truth and the predicted RIS configurations (b) and (e) display 3D-views of the radiation patterns when a  $-9$  dB mask is applied. (c) and (f) present zenithal views of the complete radiation patterns.

**Table 11**

CPS/DSP ratio comparison of the Altera® Arria® 10 evaluated architectures.

| Device            | Architecture          | $10 \times 10$ |          | $15 \times 15$ |          | $20 \times 20$ |          |
|-------------------|-----------------------|----------------|----------|----------------|----------|----------------|----------|
|                   |                       | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ |
| Altera® Arria® 10 | <b>minArea</b>        | 3,94           | 1,06     | 3,82           | 1,03     | 3,61           | 0,98     |
|                   | <b>maxPerformance</b> | 4,04           | 1,35     | 3,66           | 1,27     | 3,39           | 1,22     |
|                   | <b>Balance</b>        | 7,80           | 2,39     | 7,34           | 2,28     | 6,89           | 2,14     |

**Table 12**

CPS/(DSP+AIE) ratio comparison of the AMD Versal™ evaluated architectures.

| Device      | Architecture          | $10 \times 10$ |          | $15 \times 15$ |          | $20 \times 20$ |          |
|-------------|-----------------------|----------------|----------|----------------|----------|----------------|----------|
|             |                       | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ | $N = 25$       | $N = 50$ |
| AMD Versal™ | <b>minArea</b>        | 31,98          | 22,58    | 30,09          | 22,37    | 28,05          | 19,70    |
|             | <b>maxPerformance</b> | 18,79          | 15,63    | 17,98          | 14,84    | 17,08          | 13,60    |
|             | <b>Balance</b>        | 25,52          | 19,86    | 25,23          | 19,62    | 22,75          | 17,29    |

of the customization is the replacement of the usual SiLU layer with the Hard-SiLU, better suited for hardware. By harnessing the parallel processing capabilities inherent to FPGA architectures, it has been demonstrated how deep learning algorithms can be effectively applied in real-time to optimize the phase shifts of RIS elements, thereby improving signal quality and enhancing overall system performance with this novel approach. Moreover, this work exemplifies how this methodology can be extended to a variety of application fields beyond RIS, with FPGAs enabling deep learning acceleration in the edge-computing paradigm.

This study includes an assessment of six different deep learning networks implemented on the Altera® Arria® 10 SX SoC Development Kit and the AMD Versal™ VCK-190 Evaluation Kit, providing valuable insights into the trade-offs between performance and resource utilization. The Altera® Arria® 10, with its resource-efficient approach, provides an excellent solution for applications with moderate computational demands, achieving solid performance while maintaining low resources utilization. In contrast, the AMD Versal™, with its cutting-edge hybrid architecture that incorporates both DSP slices and AIEs, delivers significantly higher throughput, making it better suited for more demanding RIS configuration tasks that require high computational power. The

comparison of the two platforms highlights the importance of selecting the appropriate hardware based on the specific needs of the application, while the different architectures prove how it is possible to prioritize efficiency or maximizing performance by tailoring different implementation parameters. In any case, results on these platforms show how these edge implementations generate accurate predictions for the RIS configuration, both in terms of cell-prediction accuracy and radiation pattern error, while this is achieved with limited resources and notable performance.

This approach is also valid for multi-beam configurations, which would only require to first derive an appropriate dataset, since from the point of view of the deep learning architecture, multi-beam scenarios require a different set of output patterns to be associated with a different set of input angles. As output patterns are the same size as in the presented single beam scenario, the basic network architecture would be very similar but for the re-architect of the input layers, adapting them to the new number of inputs, and the fine tuning of the following layers. Additionally, the adaption of this proposal to multi-bit phase quantization does not require substantial changes to the CNN, since it outputs values that are quantized to one bit in this case, but that can be quantized to the desired multi-bit phase arrangement. Even if the network is rearchitected to have more than one output per cell, it is only necessary to modify the number of per-element outputs in the final layer. In any case, these solutions would not noticeably increase the size of the network, since the number of parameters is dominated by the previous layers.

In conclusion, the novel combination of deep learning and FPGA-based hardware presents a powerful solution for optimizing RIS configurations, which can be extended to any application on the edge. This approach not only accelerates the computation process but also offers adaptability and scalability, making it a promising avenue for future wireless communication systems. Further work can explore optimizations for power efficiency and expand the applicability of this solution to larger-scale RIS configurations, paving the way for more sophisticated, real-time wireless communication solutions.

### CRedit authorship contribution statement

**Rubén Padial-Allué:** Conceptualization, Methodology, Software, Data curation, Investigation, Writing – original draft, Writing – review & editing, Visualization. **Alberto Martín-Martín:** Conceptualization, Methodology, Software, Data curation, Investigation, Writing – original draft, Writing – review & editing. **Antonio García:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing, Visualization, Supervision. **Encarnación Castillo:** Writing – review & editing, Project administration, Funding acquisition. **José David Fernández-Rodríguez:** Data curation, Software, Writing – original draft, Writing – review & editing. **Marcos Baena-Molina:** Software, Writing – review & editing, Visualization. **Luis Parrilla:** Writing – review & editing.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgments

This work is part of the project TED2021-129938B-I00, funded by MCIN/AEI/10.13039/501100011033 and by the European Union NextGenerationEU/ PRTR and part of project PID2022-140934OB-I00 funded by MCIN/AEI/10.13039/ 501100011033/ and FEDER/UE.

Funding for open access charge: Universidad de Granada / CBUA

### Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.compeleceng.2026.111101>.

### Data availability

Data will be made available on request.

### References

- [1] Barb G-R, Oteteanu M. 5G: An overview on challenges and key solutions. In: 2018 international symposium on electronics and telecommunications, vol. 10, IEEE; 2018, p. 1–4. <http://dx.doi.org/10.1109/isetc.2018.8583916>.
- [2] Global System for Mobile Communications Association. New GSMA study: 5G to account for 15% of global mobile industry by 2025 as 5G network launches accelerate. 2024, <https://www.gsma.com/newsroom/press-release/new-gsma-study-5g-to-account-for-15-of-global-mobile-industry-by-2025/>. [Online; Accessed 05 March 2026].
- [3] Basar E, Di Renzo M, De Rosny J, Debbah M, Alouini M-S, Zhang R. Wireless communications through reconfigurable intelligent surfaces. IEEE Access 2019;7:116753–73. <http://dx.doi.org/10.1109/access.2019.2935192>.
- [4] Mu X, Xu J, Liu Y, Hanzo L. Reconfigurable intelligent surface-aided near-field communications for 6G: Opportunities and challenges. IEEE Veh Technol Mag 2024;19(1):65–74. <http://dx.doi.org/10.1109/mvt.2023.3345608>.
- [5] Ahmed M, Raza S, Soofi AA, Khan F, Khan WU, Abideen SZU, Xu F, Han Z. Active reconfigurable intelligent surfaces: Expanding the frontiers of wireless communication-a survey. IEEE Commun Surv & Tutorials 2024. <http://dx.doi.org/10.1109/comst.2024.3423460>, 1–1.

- [6] Rana B, Cho S-S, Hong I-P. Review paper on hardware of reconfigurable intelligent surfaces. *IEEE Access* 2023;11:29614–34. <http://dx.doi.org/10.1109/access.2023.3261547>.
- [7] Zhang P, Zhang J, Xiao H, Du H, Niyato D, Ai B. RIS-aided 6G communication system with accurate traceable user mobility. *IEEE Trans Veh Technol* 2023;72(2):2718–22. <http://dx.doi.org/10.1109/tvt.2022.3214818>.
- [8] Huang C, Zappone A, Alexandropoulos GC, Debbah M, Yuen C. Reconfigurable intelligent surfaces for energy efficiency in wireless communication. *IEEE Trans Wirel Commun* 2019;18(8):4157–70. <http://dx.doi.org/10.1109/TWC.2019.2922609>.
- [9] Shao X, You C, Zhang R. Intelligent reflecting surface aided wireless sensing: Applications and design issues. *IEEE Wirel Commun* 2024;31(3):383–9. <http://dx.doi.org/10.1109/MWC.004.2300058>.
- [10] Hu J, Zhang H, Di B, Li L, Bian K, Song L, Li Y, Han Z, Poor HV. Reconfigurable intelligent surface based RF sensing: Design, optimization, and implementation. *IEEE J Sel Areas Commun* 2020;38(11):2700–16. <http://dx.doi.org/10.1109/JSAC.2020.3007041>.
- [11] Jian M, Alexandropoulos GC, Basar E, Huang C, Liu R, Liu Y, Yuen C. Reconfigurable intelligent surfaces for wireless communications: Overview of hardware designs, channel models, and estimation techniques. *Intell Convergent Networks* 2022;3(1):1–32. <http://dx.doi.org/10.23919/ICN.2022.0005>.
- [12] Cui TJ, Qi MQ, Wan X, Zhao J, Cheng Q. Coding metamaterials, digital metamaterials and programmable metamaterials. *Light: Sci & Appl* 2014;3(10). <http://dx.doi.org/10.1038/lsa.2014.99>, e218–e218.
- [13] Yao H, Du X, Wang H, Gao S, Pang Y, Zheng H, Gong H, Li Y, Xu J, Zhang J, Bi K. Dual-band dynamic reconfigurable intelligent surface for signal enhancement in millimeter wave. *Sci China Technol Sci* 2024;67(4):1133–8. <http://dx.doi.org/10.1007/s11431-023-2576-0>.
- [14] Wang J, Tang W, Liang JC, Zhang L, Dai JY, Li X, Jin S, Cheng Q, Cui TJ. Reconfigurable intelligent surface: Power consumption modeling and practical measurement validation. *IEEE Trans Commun* 2024;72(9):5720–34. <http://dx.doi.org/10.1109/tcomm.2024.3382332>.
- [15] Ouyang M, Wang Y, Gao F, Zhang S, Li P, Ren J. Computer vision-aided reconfigurable intelligent surface-based beam tracking: Prototyping and experimental results. 2022. <http://dx.doi.org/10.48550/ARXIV.2207.05032>.
- [16] Yang H, Cao X, Yang F, Gao J, Xu S, Li M, Chen X, Zhao Y, Zheng Y, Li S. A programmable metasurface with dynamic polarization, scattering and focusing control. *Sci Rep* 2016;6(1). <http://dx.doi.org/10.1038/srep35692>.
- [17] Li Z, Wang S, Lin Q, Li Y, Wen M, Wu Y-C, Poor HV. Phase shift design in RIS empowered wireless networks: From optimization to AI-based methods. *Network* 2022;2(3):398–418. <http://dx.doi.org/10.3390/network2030025>.
- [18] Hassouna S, Jamshed MA, Ur-Rehman M, Imran MA, Abbasi QH. Configuring reconfigurable intelligent surfaces using a practical codebook approach. *Sci Rep* 2023;13(1). <http://dx.doi.org/10.1038/s41598-023-31596-7>.
- [19] Shan T, Pan X, Li M, Xu S, Yang F. Coding programmable metasurfaces based on deep learning techniques. *IEEE J Emerg Sel Top Circuits Syst* 2020;10(1):114–25. <http://dx.doi.org/10.1109/jetcas.2020.2972764>.
- [20] Molero C, Palomares-Caballero A, Alex-Amor A, Parellada-Serrano I, Gamiz F, Padilla P, Valenzuela-Valdés JF. Metamaterial-based reconfigurable intelligent surface: 3D meta-atoms controlled by graphene structures. *IEEE Commun Mag* 2021;59(6):42–8. <http://dx.doi.org/10.1109/MCOM.001.2001161>.
- [21] Fernández-Rodríguez JD, García-Aguilar I, Luque-Baena RM, López-Rubio E, Baena-Molina M, Valenzuela-Valdés JF. Learning to shape beams: Using a neural network to control a beamforming antenna. *Comput Netw* 2025;270:111544. <http://dx.doi.org/10.1016/j.comnet.2025.111544>.
- [22] Martín-Martín A, Padial-Allué R, Castillo E, Parrilla L, Parellada-Serrano I, Morán A, García A. Hardware implementations of a deep learning approach to optimal configuration of reconfigurable intelligence surfaces. *Sensors* 2024;24(3):899. <http://dx.doi.org/10.3390/s24030899>.
- [23] Ismail AM, Zhao Y, Wang Z, Guan YL, Yuen C. Visually steered reconfigurable intelligent surface-assisted mobile communications. *IEEE Antennas Wirel Propag Lett* 2025;1–5. <http://dx.doi.org/10.1109/lawp.2025.3584790>.
- [24] Dhillewarao P, Boppu S, Manikandan MS, Cenkramaddi LR. Efficient hardware architectures for accelerating deep neural networks: Survey. *IEEE Access* 2022;10:131788–828. <http://dx.doi.org/10.1109/access.2022.3229767>.
- [25] Hong H, Choi D, Kim N, Lee H, Kang B, Kang H, Kim H. Survey of convolutional neural network accelerators on field-programmable gate array platforms: architectures and optimization techniques. *J Real-Time Image Process* 2024;21(3). <http://dx.doi.org/10.1007/s11554-024-01442-8>.
- [26] Baena-Molina M, Galeote-Cazorla JE, Palomares-Caballero Á, Martínez-García G, Ramírez-Arroyo A, Valenzuela-Valdés JF. Indoor channel characterization for RIS-assisted 5G mmwave communications. 2025. <http://dx.doi.org/10.36227/techrxiv.176159787.70687290/v1>, TechRxiv.
- [27] Liu R, Dou J, Li P, Wu J, Cui Y. Simulation and field trial results of reconfigurable intelligent surfaces in 5G networks. *IEEE Access* 2022;10:122786–95. <http://dx.doi.org/10.1109/access.2022.3223447>.
- [28] Coskun AF, Kocoglu AT, Arslan E, Yigit Z, Kesir S, Kaplan B, Dou J, Cui Y. Experimental performances of mmwave RIS-assisted 5G-advanced wireless deployments in urban environments. 2025. <http://dx.doi.org/10.48550/ARXIV.2506.06525>, arXiv.
- [29] Trichopoulos GC, Theofanopoulos P, Kashyap B, Shekhawat A, Modi A, Osman T, Kumar S, Sengar A, Chang A, Alkhateeb A. Design and evaluation of reconfigurable intelligent surfaces in real-world environment. *IEEE Open J Commun Soc* 2022;3:462–74. <http://dx.doi.org/10.1109/ojcoms.2022.3158310>.
- [30] Chen J, Liang Y-C, Pei Y, Guo H. Intelligent reflecting surface: A programmable wireless environment for physical layer security. *IEEE Access* 2019;7:82599–612. <http://dx.doi.org/10.1109/access.2019.2924034>.
- [31] Zhang Y, Ren H, Pan C, Wang B, Yu Z, Weng R, Wu T, He Y. Secure wireless communication in active RIS-assisted DFRC systems. *IEEE Trans Veh Technol* 2025;74(1):626–40. <http://dx.doi.org/10.1109/tvt.2024.3438151>.
- [32] Cui M, Zhang G, Zhang R. Secure wireless communication via intelligent reflecting surface. *IEEE Wirel Commun Lett* 2019;8(5):1410–4. <http://dx.doi.org/10.1109/lwc.2019.2919685>.
- [33] Aubry A, De Maio A, Rosamilia M. Reconfigurable intelligent surfaces for N-LOS radar surveillance. *IEEE Trans Veh Technol* 2021;70(10):10735–49. <http://dx.doi.org/10.1109/tvt.2021.3102315>.
- [34] Zheng Y, Zheng B, Wu Q, Tang J, Zeng Y, Zhang R. Intelligent reflecting surface-enhanced radar detection with beam broadening. *IEEE Commun Lett* 2025;29(11):2711–5. <http://dx.doi.org/10.1109/lcomm.2025.3610152>.
- [35] Silva LGD, Alexandre LC, Xiao P, Cerqueira S. A. RIS development and implementation in a mm-waves 5G-NR system toward 6G. *IEEE Wirel Commun Lett* 2024;13(3):736–40. <http://dx.doi.org/10.1109/lwc.2023.3341301>.
- [36] Baena-Molina M, Palomares-Caballero Á, Martínez-García G, Galeote-Cazorla JE, Ramírez-Arroyo A, Valenzuela-Valdés JF. 1-bit mechanically reconfigurable metasurface as a beam splitter for indoor environments at 28 GHz. *IEEE Antennas Wirel Propag Lett* 2025;1–5. <http://dx.doi.org/10.1109/lawp.2025.3554271>.
- [37] Baena-Molina M, Palomares-Caballero Á, Martínez-García G, Sánchez-Zarco ÁM, Padilla P, Valenzuela-Valdés JF. A 2-bit mechanically reconfigurable intelligent surface based on stepper micromotors for 5G 26 GHz band. 2025. <http://dx.doi.org/10.36227/techrxiv.176704958.84951602/v1>, TechRxiv.
- [38] Hassouna S, Jamshed MA, Ur-Rehman M, Imran MA, Abbasi QH. RIS-assisted near-field localization using practical phase shift model. *Sci Rep* 2024;14(1). <http://dx.doi.org/10.1038/s41598-024-54859-3>.
- [39] Altera® Arria® 10 SX SoC Development Kit. 2021. <https://www.intel.com/content/www/us/en/products/details/fpga/development-kits/arria/10-sx.html>. [Online; Accessed 05 March 2026].
- [40] AMD Versal™ VCK-190 Evaluation Kit. 2024. <https://www.xilinx.com/products/boards-and-kits/vck190.html>. [Online; Accessed 05 March 2026].
- [41] AMD AI engine technology. 2024. <https://www.xilinx.com/products/technology/ai-engine.html>. [Online; Accessed 05 March 2026].
- [42] Intel® FPGA AI Suite. 2024. <https://www.intel.com/content/www/us/en/software/programmable/fpga-ai-suite/overview.html>. [Online Accessed 05 March 2026].
- [43] OpenVINO™. 2024. <https://docs.openvino.ai/>. [Online; Accessed 05 March 2026].

- [44] OpenVINO™ IR format. 2024, <https://docs.openvino.ai/2024/documentation/openvino-ir-format.html>. [Online; Accessed 05 March 2026].
- [45] Intel® FPGA AI Suite IP reference manual. 2024, <https://docs.altera.com/r/docs/768974/2023.2/intel-fpga-ai-suite-ip-reference-manual/>. [Online; Accessed 05 March 2026].
- [46] Quartus® Prime Design Software. 2024, <https://www.altera.com/products/development-tools/quartus>. [Online; Accessed 05 March 2026].
- [47] Vitis™ AI Deep-Learning Processor Unit (DPU). 2023, <https://docs.amd.com/r/3.0-English/ug1414-vitis-ai/Deep-Learning-Processor-Unit>. [Online; Accessed 05 March 2026].
- [48] Nios® V Processor for Intel® FPGA. 2026, <https://www.altera.com/products/ip/po-3098/nios-v-processors>. [Online; Accessed 05 March 2026].
- [49] Gholami A, Kim S, Dong Z, Yao Z, Mahoney MW, Keutzer K. A survey of quantization methods for efficient neural network inference. In: Low-power computer vision. Chapman and Hall/CRC; 2022, p. 291–326. <http://dx.doi.org/10.1201/9781003162810-13>.
- [50] Jain S, Gural A, Wu M, Dick C. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *Proc Mach Learn Syst* 2020;2:112–28.